15. Career Pathways and the Future of LowCode Development

Aim:

To study the application of advanced performance optimization techniques, including field indexing, subform structuring, and Deluge query tuning, in the Insurance Claim Request & Processing Management application to improve execution speed, reduce response latency, and optimize database operations

Theory:

As low-code applications grow and handle larger volumes of transactional data, inefficient data structures and poorly written Deluge scripts can cause performance bottlenecks. Zoho Creator provides several mechanisms to optimize application performance.

Field Indexing:

Field indexing improves the speed of searching, filtering, and retrieving records by creating database indexes on frequently searched fields.

Application: In the insurance claim system, fields such as Policy Number, Claimant Name, and Claim Status can be considered for indexing where supported.

Benefits:

- Faster record searches.
- Improved filtering performance.
- Reduced database lookup time.

Efficient Subform Usage:

Subforms allow related information to be maintained within a parent form. In the insurance claim application, an Investigation Details subform could store investigation dates, surveyor details, findings, and investigation status.

Benefits:

- Organizes related information.
- Simplifies data entry.
- Improves management of investigation records.

3. Deluge Query Tuning

Deluge scripts can be optimized by using precise criteria, limiting the number of fetched records, and avoiding unnecessary database operations inside loops.

For example, retrieving only submitted claims instead of fetching every claim can reduce unnecessary processing.

Benefits:

- Reduces database operations.
- Improves script execution speed.
- Helps avoid execution timeouts.

Procedure:

Step 1: Identifying Fields for Indexing

- Open the Insurance Claim Request & Processing Management application in Zoho Creator.
- Open the Claim Request Form in the Form Designer.
- Identify frequently searched fields, such as Policy Number and Claim Status.
- Open the field properties and enable indexing where supported.
- Save the configuration.

Step 2: Structuring Subforms

- Open the Claim Request Form in the Form Designer.
- Add an Investigation Details subform if multiple investigation entries need to be maintained for each claim.
- Include fields such as Investigation Date, Surveyor Name, Findings, and Investigation Status.
- Configure the necessary field properties and save the form.

Step 3: Optimizing Deluge Queries

- Open the Script Builder for the relevant form events or workflows.
- Review existing Deluge scripts for unnecessary database queries.
- Use specific criteria to retrieve only relevant claim records.
- Limit fetched records when processing large datasets.
- Avoid repeated database operations inside loops.
- Save and test the optimized scripts.

Example Deluge query:

```
claims = Claim_Request_Form[Claim_Status == "Submitted"];
```

Step 4: Testing Performance

- Open the Application Debugger and execution logs, where available.
- Test form submissions and report loading before and after optimization.
- Compare search speed and script execution time.
- Verify that the optimized queries return the correct records.
- Confirm that existing workflows and validations continue to work.

Result

Advanced performance optimization techniques were studied for the Insurance Claim Request & Processing Management application. Field indexing can improve search performance, subforms can organize related investigation data, and Deluge query tuning can reduce unnecessary database operations.